

DATA VALIDATION ENGINEER · MASTERY PATH

Pentaho Mastery for a Data Validation Engineer

From zero to reading any .ktr / .kjb, proving the target with SQL, talking to ETL people.

Parts A-H · mental model through senior questions

Spoon, steps, jobs, streaming internals

DVE loop, six checks, Jira bug template

Never-do list, standup script, 7 senior questions

Video index with full YouTube URLs

Study sheet · Keep this next to Spoon / MySQL / Jira

How to use this guide

Read it in order. You do not become a Pentaho developer on day 1. You become someone who can open any pipeline, understand what it did to the rows, prove the target with SQL, and talk to ETL people without blinking.

What you will actually touch: Pentaho Data Integration (PDI), also called Kettle. The desktop is Spoon.

What you will not live in: Pentaho reports, dashboards, BA server. Those exist. They are not your job.

Your proof tool stays MySQL. Pentaho tells you how data moved. SQL proves what landed. Jira is where the defect lives.

Teach yourself in this order:

- Part A — Mental model (ETL kitchen, seven words, product names)
- Part B — Spoon like a native (UI, files, Preview vs Run, connections)
- Part C — Step types a master must recognize
- Part D — Jobs like a master (orchestration, hops, fail paths)
- Part E — How data actually moves (streaming, counts, variables)
- Part F — The Data Validation Engineer loop (this is the job)
- Part G — Sit with senior ETL people (facts, SCD, CDC, dictionary)
- Part H — Practice path, standup script, seven senior questions
- Video index — every YouTube URL in one table

PART A — Mental model (absolute zero)

A1. What ETL is

Imagine a restaurant kitchen.

- Extract = take ingredients out of the crates (source files / source tables).
- Transform = wash, trim, cut, throw away spoiled pieces, mix to the recipe (mapping rules).
- Load = plate it and send it to the dining room (target warehouse table).

The recipe is the source-to-target mapping (STM). If it says “trim and uppercase member_id, drop VOID claims,” the kitchen must do exactly that. Your job is not to invent a new recipe. Your job is to taste the plate and say: this matches the recipe, or it does not.

ETL is a scheduled process that copies data from sources, applies business rules, and writes a clean result into a target (usually a warehouse). Reports then query that, not messy operational tables.

You already know this as SQL:

```
INSERT INTO dw.claims_fact (claim_id, member_id, status, amount)
SELECT claim_id, UPPER(TRIM(member_id)), status, amount
FROM src.claims
WHERE status <> 'VOID';
```

Pentaho does the same work as boxes and arrows instead of one script. STM is the recipe. DVE tastes the plate.

Watch What is ETL? Explained Simply <https://www.youtube.com/watch?v=5zRz6uXzHIU>

Definition in plain language. Start here if ETL is still a blur.

Watch What is ETL? — IBM <https://www.youtube.com/watch?v=OW5OgsLpDCQ>

Why companies bother (~5 min).

A2. Why not “just SQL”?

1. Many sources — CSV, Excel, Oracle + MySQL + an FTP file in one flow.
2. Visible logic — a Filter box is easier to review than a 400-line query.
3. Reuse — one lookup used by twenty loads.
4. Operations — wait for a file, email on fail, retry, run at 2 a.m.

5. You can see the mistake — “Where did VOID rows die? Which hop? Does that match the STM?”

A3. Product family vs what you touch

Name	What it is	Do you care?
Pentaho	Product family	Brand name
PDI	Data Integration — the ETL engine	This is your tool
Kettle	Old name for PDI	Yes. .ktr = Kettle TRansformation
Spoon	Desktop designer	You live here
Pan	CLI that runs a transformation	Overnight runs
Kitchen	CLI that runs a job	Schedulers use this
Carte	Server that runs jobs remotely	High level only
Hitachi	Current owner / branding you may hear	Name only — still PDI
Reports / dashboards	Other Pentaho products	Not your day job

A4. Seven words you must never mix up

1. Transformation (.ktr) — the conveyor that moves and changes rows. Steps run in parallel. Data streams through.
2. Job (.kjb) — the checklist. “Wait for file → run A → run B → email if fail.” Entries run in order. Jobs do not reshape columns.
3. Step — one box. Table Input, Filter Rows, Table Output.
4. Hop — the arrow. In a job, it can be unconditional / true / false.
5. Repository — shared cupboard of jobs (files or a metadata DB). Not the warehouse.
6. Pan — CLI for a .ktr.
7. Kitchen — CLI for a .kjb.

Memory hook: .ktr = the cooking. .kjb = the shift manager.

Watch Understanding Pentaho Data Integration (PDI) — Edureka <https://www.youtube.com/watch?v=J8NbYQaQiPo>

Best single overview (~92 min). Watch 0:00–30:00 tonight if exhausted.

Watch	Why
First ~20 min	PDI vs Kettle, Spoon / Pan / Kitchen / Carte, job vs transformation
Spoon layout + a sample	Left Design/View, canvas, hops
Preview + green Run + Step Metrics	How you inspect a pipeline
Command-line Pan (late)	How QA/Prod actually run overnight

PART B — Spoon like a native

Spoon is an IDE for pipelines. Visual Studio for data movement.

B1. What you see

- Left — Design: folders of steps (Input, Transform, Output, Flow, Scripting, Lookup, Job).
- Left — View: steps and hops on this file, plus connections.
- Center: boxes and arrows (the canvas).
- Top — green play: Run. Also Preview, Debug, Stop.

Bottom after a run — Execution Results:

- Step Metrics — read / written / errors / speed per step. Your first truth.

- Logging — errors, SQL the step fired.
- Performance — timing.

A green tick means that step did not throw. It does not mean the data is correct.

B2. Files

File	Opened as	You ask
TR_*.ktr e.g. TR_CLAIMS_FACT.ktr	Transformation	What happens to each row?
JB_*.kjb e.g. JB_LOAD_CLAIMS.kjb	Job	In what order? What on fail?

Both are XML. You read the picture, not the XML.

B3. Preview vs Run

- Preview (right-click a step): see N rows at that point, usually without writing the target.
- Run: the whole transformation. Table Output will write.

DVE rule: Preview to understand. SQL on the target to certify. Never certify from Preview alone.

B4. Connections — never Prod

A connection is a JDBC login. The same .ktr can point at Dev, QA, or Prod. Day-1 sentence: “I’ll open this in Spoon on the QA connection. I won’t run anything that writes to Prod.”

If someone says “just run it,” ask: which connection, and is Table Output truncate or append?

Watch Pentaho How-to: Building Data Pipelines <https://www.youtube.com/watch?v=ID2zumlyt9Q>

Hops + inspect data.

Watch Mastering PDI Spoon UI — My First ETL Job <https://www.youtube.com/watch?v=m5GTZ5TO1UM>

First job skeleton in Spoon.

Watch Install Pentaho PDI on Windows https://www.youtube.com/watch?v=O13p5S4B_Eg

Optional — only if you need to install locally.

PART C — Step types a master must recognize

A stream is the flowing set of rows (the current result set).

C1. Input (Extract)

Table Input — SELECT ... FROM src.claims WHERE load_dt = Look at: the SQL, filters, \${LOAD_DT}, which connection.

- Bugs: extract filter ≠ STM; SELECT * then drop columns later; filter status here and later (double filter).
- Your SQL: copy their Table Input SQL, run it, COUNT(*). That is your source count after the extract filter.

CSV / Text file input — delimiter, enclosure, header, encoding (UTF-8 vs Windows-1252), date format.

- Bugs: extra column shifts fields; NULL vs empty; yesterday’s file still in the folder.

Excel Input — sheet name, header row, empty rows at the top.

Generate Rows — fake data for a test kitchen. If you see it in a real load, ask why.

Watch Demonstration of a PDI transformation <https://www.youtube.com/watch?v=sAhKX2tzVIU>

File in → steps → out. Pause when they open a step and read the fields.

C2. Transform (the recipe)

Step	What it does / what you watch
Select Values	SELECT col AS new_name plus drop/rename/type. Watch String vs Integer vs Date and truncated lengths.
Filter Rows	WHERE. True path and False path. Where do false rows go — discarded or error file? Inverted condition; numbers as strings; NULLs fail and vanish. If in = out on Step Metrics, the filter did nothing.
Calculator / Formula	amount * 1.18, date math. Watch rounding, integer division, tax applied twice.
Modified JavaScript Value	Hidden script. Read it. Magic and silent bugs live here.
Regex / Split / Concat	SUBSTRING, CONCAT, REGEXP. Split on comma while addresses contain commas.
Unique rows	SELECT DISTINCT after a Sort. Unique without Sort is wrong.
Sort rows	ORDER BY. Required before Unique, Merge join, Merge rows (diff).
Group by	Rows collapse. 1,000,000 in → 50,000 groups can be correct. Check grain (claim vs claim-line).
Normaliser / Denormaliser	Unpivot / pivot.
Add constants / Get System Info	Hardcoded yesterday's date left in is a classic bug.

Watch Filter Rows in PDI <https://www.youtube.com/watch?v=j-mCojDa-gw>

C3. Lookup / join (where rows silently disappear)

This is the #1 place “the job succeeded” lies to you.

Step	Meaning / trap
Database lookup	For each row, SELECT ... FROM dim WHERE key = ?. Watch “do not pass the row if lookup fails.” That is an inner join. Unmatched keys vanish.
Stream lookup	Small reference table in memory. If the lookup stream is not unique, you get the wrong first match.
Merge join	SQL join, but both sides must be sorted. Watch INNER vs LEFT.
Merge rows (diff)	Compare yesterday vs today (identical / changed / new / deleted). If they compare a timestamp that always changes, everything looks “changed.”
Dummy	A sink. If Dummy is the only destination of the false Filter path, those rows are thrown away.

C4. Output (Load)

Step	What you check
Table output	INSERT. Truncate vs append, commit size, field mapping. Truncate on an incremental job is a P1.
Insert / Update	Upsert. Wrong keys → duplicates or updates the wrong person.
Update / Delete	Confirm they are allowed in QA. Confirm keys.
Text / Excel writer	Path (shared QA folder vs C:\Users\...), overwrite, delimiter.

C5. Flow

- Switch / case — CASE, multiple hops.
- Block this step until steps finish — wait for slower branches.
- Abort — kill the transformation (e.g. 0 rows is not acceptable).
- Mapping (sub-transformation) — call another .ktr like a function. Open the child.

C6. Error handling

PDI can attach an error hop (often red). Bad rows go to a dump instead of failing the job. Ask every time the happy-path count is short: reject file? error table? Write to log hiding errors?

Data Validator — not null, type, range, list. Failures abort or error-hop? Your STM checks should include those rules plus the ones they forgot.

C7. Dimension Lookup / Update (SCD)

- Type 1: overwrite. History dies. Count stays ~stable.
- Type 2: new row, old row closed (end_date / current_flag = 'N'). Row count grows. Same member_id can exist twice with different surrogate keys.

Bugs: Type 2 on a field that flaps every load → dimension explodes; facts point at expired surrogate keys; two Y current flags for one business key.

Watch SCD explained with real examples <https://www.youtube.com/watch?v=XqdZF0DJpUs>

Type 1 and Type 2 only, learn this cold.

C8. Job ↔ transformation

Copy rows to result / Get rows from result — a job loops a transformation per file or country. Validate once per loop value, not only the last one.

PART D — Jobs like a master

A job is a ServiceNow workflow. Entries are catalog tasks. Typical skeleton:

```
Start → Wait for file → SQL (truncate stage)
      → Transformation → Transformation → Success
false hop → Mail → fail
```

Entry	Meaning	You ask
Start	Only one	Anything disconnected?
Transformation	Runs a .ktr	Which file? Which parameters?
Job	Nested job	Open the child
Success / Abort / Fail	End states	Abort on zero rows?
Mail	Email	Who? Is fail silent?
Wait for file	Poll a folder	Timeout? Stale file from yesterday?
SQL	Truncate, index rebuild	Read it. Wrong table = P1
FTP / Copy files	File movement	Archive? Delete after?
Set variables	\${LOAD_DT}, \${ENV}	QA vs Prod values?

Job hops are not data hops

- Unconditional — always next (even if previous failed). Dangerous after a load.
- Follow when true — previous succeeded.
- Follow when false — previous failed → usually Mail.

If a failed Transformation still runs the next load, that is a “partial warehouse” incident. When a night job fails, ask: Kitchen log or Control-M / Jenkins log? Step name? First error line? Then open that .ktr.

Watch Spoon UI & first ETL job <https://www.youtube.com/watch?v=m5GTZ5TO1UM>

Then Edureka's Kitchen vs Pan segment in the PDI overview.

Watch Edureka PDI — Kitchen vs Pan <https://www.youtube.com/watch?v=J8NbYQaQiPo>

Same overview video; jump to the CLI / Kitchen vs Pan segment.

PART E — How data actually moves

A transformation is a streaming, parallel pipeline.
It is not “finish step 1 on the whole table, then start step 2.”

Each step is a thread. Rows pass in batches. Preview of a middle step can show data while Table Output is still writing.

After this step	Count usually...	Why
Table Input	= extract SQL	Source baseline
Filter (discard path)	down	Rows dropped
Unique / Group by	down	Rows collapsed
Normaliser	up	Unpivot
Lookup “don’t pass if fail”	down	Silent inner join
Merge join INNER	down (or explode)	Join math
Merge join LEFT	same as left, unless...	Duplicates on right explode
Table Output	= inserts attempted	Errors may reject

Many-to-many join explosion: 1.2M source rows → 8M target rows. Always check uniqueness of lookup keys.

Commit size / partial load: if a job dies at 800k of 1.2M, partial commit may have loaded 800k. Count this batch id / load_dt, not the whole table forever.

Lazy conversion: a bad date can survive until Calculator / Table Output, then fail. Preview looks fine; the run dies at the last step.

Thing	Looks like	Set where
Variable	\${LOAD_DT}	kettle.properties, Set variables, Kitchen -param:
Named parameter	on the .ktr / .kjb	Run dialog, Kitchen CLI

Wrong \${LOAD_DT} = you validated the wrong day. QA box vs your laptop can differ.

How it runs at night: Spoon (local) · Pan (.ktr) · Kitchen (.kjb) · Carte (remote). File repository vs DB repository — ask which cupboard they use. Ask for the log of the run you are certifying.

PART F — The Data Validation Engineer loop

F1. The STM is the bible

Source	Target	Rule
src.claims.member_id	dw.claims_fact.member_id	UPPER(TRIM())
src.claims.status	—	exclude VOID

If the .ktr disagrees with the STM, that is a bug (or a stale STM — still raise it).

F2. Walk a .ktr like a code review

1. Connection — QA?
2. Table Input SQL — copy it.
3. Every Filter / JavaScript / Calculator — write the rule in English.
4. Every lookup — inner or left? unique keys?
5. Table Output — table, truncate?, field map.
6. Error hops / reject files.
7. Child mappings — open them.

8. Variables — which load date?

Then open the parent .kjb: order, truncate, mail on fail.

F3. Preview vs full run vs SQL

Method	Good for	Not enough for
Preview	understand a step	sign-off
Execution Results	row accounting	value correctness
SQL on target	certify	—

F4. The six checks — mapped to Pentaho

1. Count — source = Table Input SQL (same filters). Target = output table + same load_dt. Short? Filter, lookup-fail-drop, Unique, or reject hop.
2. Duplicates — GROUP BY key HAVING COUNT(*) > 1. Cause: missing Unique, many-to-many lookup, Type 2 without current_flag, wrong upsert key.
3. Nulls — required columns from STM. Cause: failed lookup that passes empty fields; Select Values dropped a column; blank CSV.
4. Transforms — re-apply the rule in SQL: UPPER(TRIM(s.x)) = t.x.
5. Orphans — LEFT JOIN dim WHERE dim.key IS NULL. Cause: lookup drop, or fact loaded before dimension job.
6. Sample — 10–20 keys, field by field. Catches what averages out in SUM.

Count / dupes / nulls / transforms / orphans / sample — map each to the hop that could have caused it.

Watch ETL Testing Part 1 — SDET-QA https://www.youtube.com/watch?v=bYgd_OBfGRE

Your profession, not Spoon. Watch 0:00–40:00 — warehouse vs OLTP, why SQL is the weapon.

Watch Common test cases for ETL Testing <https://www.youtube.com/watch?v=TiHHBLi7N2I>

F5. Success ≠ correct

Green ticks mean: no exception. They do not mean counts, transforms, or orphans are right.

"Kitchen finished success. I'm still running count and transform checks for load_dt 22-Aug. Success isn't certification."

F6. Jira

Log a Bug, not a chat. Title template:

[Data] claims_fact count short after JB_LOAD_CLAIMS / TR_CLAIMS_FACT — source 1,204,331 vs target 1,204,108 (22-Aug).
Filter on status='VOID' in Filter Rows; VOID not in STM.

Body fields:

- Job name and .ktr file
- Step name and hop (true / false / error)
- Connection (QA — never Prod)
- Batch id / load_dt
- SQL you ran, expected vs actual
- Execution Results (Step Metrics screenshot or numbers)
- Reject file / error table if any

Ready for Retest = ServiceNow Resolved-awaiting-you. Retest the same load date / same SQL. Then Done.

F7. Never

Never do these

- Run a write against Prod.
- "Just flip a hop to see."
- Close a bug you did not retest.
- Compare raw source to target without the job's filters.
- Edit a shared .ktr as an experiment.
- Certify from Preview.

PART G — Sit with senior ETL people**Fact vs dimension vs surrogate key**

- Dimension: who / what / where (member_dim). Changes slowly.
- Fact: events / measures (claims_fact). Keys point at dimensions.
- Business key: member_id from source.
- Surrogate key: member_sk invented by the warehouse. Facts store this.

Validate: every fact member_sk exists in dim and (Type 2) is the version valid on the claim date.

Incremental vs full refresh

Load	What happens	How you validate
Full refresh	Truncate + reload	Target count = source after rules
Incremental	New/changed since watermark	Today's batch = source delta; old days unchanged

Ask: "Is JB_LOAD_CLAIMS incremental? What's the watermark column?"

Prove old days unchanged:

```
SELECT load_dt, COUNT(*), SUM(amount)
FROM dw.claims_fact
WHERE load_dt < CURDATE()
GROUP BY load_dt
ORDER BY load_dt;
-- yesterday's groups must match the certified snapshot
```

CDC

Change Data Capture — the job reads inserts / updates / deletes instead of the whole source. STM must say what a source delete becomes on the target.

Performance

Slow usually means Sort on millions, Database lookup with cache off (one SQL per row), cartesian lookup, tiny commit size. Step Metrics ≠ MySQL EXPLAIN. Don't mix the words. Carte is the remote runner — high level only unless they ask you to read a Carte log.

PDI vs Informatica vs SSIS

PDI	Informatica	SSIS	Meaning
Transformation .ktr	Mapping	Data Flow	Row pipeline
Job .kjb	Workflow	Control Flow	Orchestration
Step	Transformation	Transform	One box

PDI	Informatica	SSIS	Meaning
Hop	Link	Connector	Arrow
Spoon	Designer	Visual Studio	GUI
Kitchen / Pan	pmcmd	dtexec	CLI
Filter Rows	Filter	Conditional Split	WHERE
Dimension Lookup/Update	SCD	Slowly Changing Dimension	History

If someone says “check the mapping,” they might mean the Informatica mapping or the STM spreadsheet. Ask which.

Watch SCD with real examples (again when you reach this) <https://www.youtube.com/watch?v=XqdZF0DjpUs>

PART H — Practice path

Tonight (~90 min)

1. ETL in 60 seconds — <https://www.youtube.com/watch?v=5zRz6uXzHIU>
2. Edureka PDI 0:00–30:00 — <https://www.youtube.com/watch?v=J8NbYQaQiPo>
3. Building Data Pipelines once — <https://www.youtube.com/watch?v=ID2zumlyt9Q>
4. On paper: job vs transformation; the six SQL checks; one Jira bug template

Day 1

Get the board, STM, first QA job name, Spoon access, QA connection. Open the .kjb, then the .ktr. Write the English story of the hops. Run counts only on one table.

Week 1

Filter + lookup + Table Output on every assigned job. One Type 2 dimension: prove current_flag uniqueness. Then Filter Rows (<https://www.youtube.com/watch?v=j-mCojDa-gw>) and ETL Testing Part 1 (https://www.youtube.com/watch?v=bYgd_OBfGRE).

Month 1 (mastery starting)

Open an unknown .ktr and predict count change at each hop before SQL. Catch a silent inner-join lookup. Certify incremental without wrecking history.

What master sounds like in standup

“JB_LOAD_CLAIMS Kitchen success on QA. TR_CLAIMS_FACT Table Input 1,204,331 after load_dt filter. Filter Rows drops VOID — 223 rows — not in STM, logged PAY-1842. Target 1,204,108. Lookup on member is left-join, 0 orphans. I’ll retest PAY-1842 when they remove the VOID filter or the STM is updated.”

Questions that make seniors take you seriously

1. Is the member lookup fail-pass or fail-drop?
2. What’s the grain of claims_fact — header or line?
3. Incremental watermark column, and do we reprocess late-arriving rows?
4. Type 2 fields on member_dim — which attributes close a version?
5. Where do reject rows go, and are they in the count SLA?
6. Which Kitchen log is source of truth — Control-M or PDI logging table?
7. Do facts join dim on surrogate + effective date, or current flag only?

Video index — full URLs (clickable)

Type the https URL if the link does not open from your PDF reader. All videos used in this guide:

Concept	YouTube URL
What ETL means (simple)	https://www.youtube.com/watch?v=5zRz6uXzHIU
Why ETL exists (IBM)	https://www.youtube.com/watch?v=OW5OgsLpDCQ
PDI / Spoon / Pan / Kitchen (Edureka)	https://www.youtube.com/watch?v=J8NbYQaQiPo
Hops, inspect data, Table Output	https://www.youtube.com/watch?v=ID2zumlyt9Q
First job: Start → Transformation → Success	https://www.youtube.com/watch?v=m5GTZ5TO1UM
Full file transformation demo	https://www.youtube.com/watch?v=sAhKX2tzVIU
Filter Rows true/false	https://www.youtube.com/watch?v=j-mCojDa-gw
Install (only if needed)	https://www.youtube.com/watch?v=O13p5S4B_Eg
SCD types	https://www.youtube.com/watch?v=XqdZF0DJpUs
ETL testing mindset (0–40)	https://www.youtube.com/watch?v=bYgd_OBfGRE
ETL test cases list	https://www.youtube.com/watch?v=TiHHBLi7N2I

Closing

You do not need to build pipelines tomorrow. You need to read them, account for every row, and prove the plate against the recipe.

Spoon is the kitchen window.

MySQL is the lab test.

Jira is the incident.